

SOFA

Coffee Shop & Bakery

Customer Ordering System

D2: Requirements Report

CSE323 Software Engineering · Spring 2026

Member Slice	Responsibility
Member 1	Menu browsing, filtering, and item visibility
Member 2	Cart state, quantity validation, and totals
Member 3	Order placement, checkout validation, and order creation
Member 4	Authentication, session continuity, and order history
Member 5	Order tracking, status timeline, and status auto-progression

Team Member	Student ID
Shahd Hamdy Abd Ellah	120230070
Ali Ahmed El zwawy	120230058
Ahmed Salah Elsayed	120230011
Omar Sayed Darder	120230018
Ahmed Osama Ahmed	120230014

Submission Deadline: 18 May 2026

Contents

1	System Overview	2
2	Actor Classification	2
2.1	Primary Actors (initiate use cases)	2
2.2	Supporting Actors (provide services to the system)	2
2.3	Offstage Actors (have a stake but never interact directly)	2
3	Functional Requirements	2
3.1	Menu Requirements	3
3.2	Cart Requirements	3
3.3	Order Placement Requirements	4
3.4	Authentication & Order History Requirements	4
3.4.1	Login & Registration	5
3.4.2	Login & Registration — UI Requirements	6
3.4.3	Order History	6
3.4.4	Authentication & Order History — API Endpoints	7
3.5	Order Tracking Requirements	7
3.6	Non-Functional Requirements	8
4	Traceability Heatmap	8
5	Persona Discovery, AI User Avatar Exercise	10
5.1	Impact of Persona Discovery on Requirements	12
A.	AI Disclosure	12

1 System Overview

Sofa is a coffee shop and bakery ordering system that enables customers to browse a digital menu, build a cart, place an order, and track its preparation status in real time. The system replaces manual counter-based ordering with an automated web interface accessible from any browser-enabled device.

The system is scoped to the Customer Ordering sub-system, which spans five functional slices: Menu, Cart, Order Placement, Authentication & Order History, and Order Tracking. Order status is managed automatically by time-based progression (each stage advances every 2 minutes); no staff-facing dashboard is included in this scope.

2 Actor Classification

Following the three-actor taxonomy (Primary, Supporting, Offstage), we identify all system participants.

2.1 Primary Actors (initiate use cases)

Primary actors directly interact with the system to achieve personal goals.

- **Customer:** The sole system user. Browses the menu, manages their cart, places orders, monitors order status, and reviews personal order history. Every core use case is initiated by a customer.

2.2 Supporting Actors (provide services to the system)

- **Flask Back-end API:** Handles all business logic, session management, and database operations.
- **SQLite Database Engine:** Persists menu items, cart state, orders, and tracking events.
- **Web Browser (HTTP Client):** Renders the HTML/CSS/JS frontend and communicates with the backend via fetch/AJAX requests.
- **Session Manager:** Manages customer session cookies to associate anonymous cart and order data with a browser session without requiring user authentication.

2.3 Offstage Actors (have a stake but never interact directly)

- **Shop Owner / Manager:** Requires accurate sales data and order throughput metrics; no management analytics feature is in scope.
- **Health & Food Safety Regulator:** Mandates that allergen and dietary information be accurate and available, creating a hidden requirement on the menu data model.
- **Accessibility Auditor:** Web interfaces must meet minimum WCAG 2.1 AA contrast and keyboard-navigation requirements, influencing the frontend CSS design.

3 Functional Requirements

All requirements are identified with a unique ID of the form F-XXX-NN (Feature area – sequence number).

3.1 Menu Requirements

ID	Requirement	Priority
F-MNU-01	The system shall display all menu items grouped by category (e.g. Coffee, Pastries, Cold Drinks).	High
F-MNU-02	Each menu item shall show name, price (in EGP, 2 decimal places), short description, allergen tags, and an availability indicator.	High
F-MNU-03	The system shall filter menu items by category without a page reload.	High
F-MNU-04	Unavailable items shall be displayed with a visual “sold out” indicator and the Add-to-Cart button disabled.	Medium
F-MNU-05	The system shall respond to a GET /api/menu request within 500 ms under normal load (50 concurrent users).	Low
F-MNU-06	Menu data shall be stored in an items table; the API shall never expose internal database row IDs in a predictable sequence.	High

Table 1: Menu functional requirements

3.2 Cart Requirements

ID	Requirement	Priority
F-CRT-01	The system shall maintain a session-based cart that persists across page refreshes for the same browser session.	High
F-CRT-02	Adding an item already in the cart shall increment its quantity rather than creating a duplicate entry.	High
F-CRT-03	The system shall allow customers to increase, decrease, or remove individual cart items.	High
F-CRT-04	The cart shall display a real-time subtotal computed as $\sum_i (qty_i \times price_i)$, rounded to 2 decimal places.	Medium
F-CRT-05	Cart item quantity shall not exceed 20 units per item per session; values outside $[1, 20]$ return HTTP 422.	High
F-CRT-06	The system shall prevent adding an unavailable item to the cart, returning HTTP 409 if attempted via the API.	High

Table 2: Cart functional requirements

3.3 Order Placement Requirements

ID	Requirement	Priority
F-ORD-01	The system shall allow an authenticated customer to place an order from a non-empty cart by providing a customer name (1–60 characters).	High
F-ORD-02	On successful order placement, the system shall clear the cart and return a unique order id (UUID v4). Duplicate POST /api/order requests within a 5-second idempotency window shall return the existing order id rather than creating a new order.	High
F-ORD-03	The system shall reject order placement if the cart is empty, returning HTTP 400.	High
F-ORD-04	The stored order total shall equal the exact sum of (item price $\times$ quantity) at the time of placement; price changes after placement shall not affect existing orders.	Medium
F-ORD-05	The system shall assign each new order an initial status of pending.	High
F-ORD-06	Customer name shall not contain HTML or script tags; input must be sanitised server-side and match $/^{\{L\}' - .\}\{1, 60\}\$/u$.	High

Table 3: Order placement functional requirements

3.4 Authentication & Order History Requirements

3.4.1 Login & Registration

ID	Requirement	Priority
F-AUTH-01	The system shall allow a new customer to register by providing a unique email address, a unique username, a full name, and a password (minimum 8 characters, validated client- and server-side).	High
F-AUTH-02	Passwords shall never be stored in plain text; the system shall hash them using <code>werkzeug.security.generate_password_hash</code> before persisting to the database.	High
F-AUTH-03	A registered customer shall be able to log in with their email and password. On success, the server shall issue a signed session token (JWT) stored client-side.	High
F-AUTH-04	A logged-in customer's session shall persist across page reloads for the token's lifetime (default: 24 hours). The token shall be stored in <code>localStorage</code> .	High
F-AUTH-05	The customer shall be able to log out, which clears the client-side token.	High
F-AUTH-06	The history page (<code>/history.html</code>) shall be accessible only to authenticated customers; unauthenticated access shall redirect to the login page.	Medium
F-AUTH-07	The shared navbar shall reflect the customer's auth state: displaying a "Login" link when unauthenticated and the customer's name with a "Logout" button when authenticated.	Low
F-AUTH-08	(Optional) When a customer logs in, the system should attempt to associate any orders placed under the current session id with their account, provided those orders were placed within the same browser session.	Low

Table 4: Authentication functional requirements

3.4.2 Login & Registration — UI Requirements

ID	Requirement	Priority
F-AUTH-09	The login page (login.html) shall contain: an email input, a masked password input, a “Login” submit button, and a link to the registration page.	High
F-AUTH-10	The registration page (register.html) shall contain: a full name input, a unique username input, an email input, a password input, a confirm-password input, and a “Register” submit button.	High
F-AUTH-11	Form fields shall display inline error messages for: missing required fields, invalid email format, password under 8 characters, and mismatched confirm-password.	High
F-AUTH-12	If authentication fails, the page shall display a single generic error: “Invalid email or password.” The system shall not distinguish which field was incorrect (security requirement).	Medium
F-AUTH-13	After a successful login or registration, the customer shall be redirected to the history page (or the originally requested protected page, if available).	Medium

Table 5: Authentication UI requirements

3.4.3 Order History

ID	Requirement	Priority
F-HST-01	The history page shall display all orders associated with the authenticated customer’s account, sorted by created_at descending (most recent first).	High
F-HST-02	Each order card shall display: order reference (order_public_id, truncated), order date/time (DD/MM/YYYY HH:MM), a colour-coded status badge, total amount, and a “View Details” toggle.	High
F-HST-03	Expanding an order shall reveal individual items, showing item name, quantity, unit price, and line total.	Medium
F-HST-04	If the customer has no orders, the page shall display a friendly empty-state message with a link back to the menu.	Medium
F-HST-05	If the customer has more than 10 orders, results shall be paginated (10 per page) with Previous/Next navigation.	Low
F-HST-06	(Optional) A “Re-order” button on each history card may allow the customer to re-add all items from a past order to their current cart with a single click.	Low

Table 6: Order history functional requirements

3.4.4 Authentication & Order History — API Endpoints

Method	Endpoint	Description
POST	/api/auth/register	Registers a new customer account
POST	/api/auth/login	Authenticates and returns a session token
POST	/api/auth/logout	Clears the current session
GET	/api/auth/me	Returns the authenticated customer's profile
GET	/api/history	Paginated order history (auth required)
GET	/api/history/<order_public_id>	Full item breakdown for one past order (auth required)

Table 7: Authentication and history API endpoints

Acceptance Criteria — Member 4:

- A customer cannot register with an email that already exists; the API returns 409 Conflict.
- Login with an incorrect password returns 401 with the generic error message.
- The history page is unreachable without a valid token; the client redirects to the login page.
- Order history is scoped strictly to the authenticated customer; one customer cannot view another's orders.
- Orders are listed most-recent-first by default.
- Each order card correctly displays all items and the correct total.
- Logout clears the token from localStorage and redirects to the login page.

3.5 Order Tracking Requirements

Order status progresses automatically through four fixed stages. Each stage has a fixed duration of 2 minutes, giving a total completion time of 6 minutes from order placement:

pending (0 min) → preparing (2 min) → ready (4 min) → completed (6 min)

ID	Requirement	Priority
F-TRK-01	A customer shall track their order by entering their order id (UUID) and see the current status and a visual progress bar. Tracking results shall expose only status and timestamp — not customer name or item breakdown — to anonymous callers.	High
F-TRK-02	The tracking page shall poll GET /api/track/:id every 10 seconds without a full page reload. Polling shall stop once the order reaches completed status.	High
F-TRK-03	The system shall return HTTP 404 for a tracking request referencing a non-existent order.	High
F-TRK-04	Every status change shall be logged to a tracking_events table with a UTC timestamp; the tracking page shall display the full chronological timeline with timestamps in the customer's local timezone.	High
F-TRK-05	The tracking page shall display an estimated wait time in minutes. The total completion time is fixed at 6 minutes, with each stage lasting exactly 2 minutes. Remaining time = $\max(0, 6 - \text{elapsed_minutes})$, rounded up to the nearest minute.	Medium

Table 8: Order tracking functional requirements

3.6 Non-Functional Requirements

ID	Requirement	Priority
NF-01	All API responses shall include a Content-Type: application/json header.	High
NF-02	The frontend shall render correctly on Chrome, Firefox, and Safari (latest stable versions).	Medium
NF-03	All HTML pages shall achieve a minimum contrast ratio of 4.5:1 for normal text (WCAG 2.1 AA).	High
NF-04	The system shall handle concurrent requests from up to 50 sessions without data corruption (session isolation).	High
NF-05	All SQL queries shall use parameterised statements; no raw string interpolation is allowed.	High
NF-06	The system shall not store customer payment information (out of scope).	High

Table 9: Non-functional requirements

4 Traceability Heatmap

The matrix below maps every functional requirement to the system component or feature that implements it. Colour intensity indicates implementation priority. No requirement may appear without at least one implementing component (no orphaned requirements).

Legend: H = High, M = Medium, L = Low, — = Not applicable

Req. ID	Menu	Cart	Order Place.	Auth & Hist.	Tracking	Shared / DB
F-MNU-01	H	—	—	—	—	L
F-MNU-02	H	—	—	—	—	L
F-MNU-03	H	—	—	—	—	—
F-MNU-04	H	H	—	—	—	L
F-MNU-05	M	—	—	—	—	M
F-MNU-06	L	—	—	—	—	H
F-CRT-01	—	H	—	—	—	H
F-CRT-02	—	H	—	—	—	H
F-CRT-03	—	H	—	—	—	—
F-CRT-04	—	H	—	—	—	—
F-CRT-05	—	M	—	—	—	—
F-CRT-06	H	H	—	—	—	H
F-ORD-01	—	H	H	—	—	—
F-ORD-02	—	H	H	—	—	H
F-ORD-03	—	H	H	—	—	—
F-ORD-04	H	H	H	—	—	H
F-ORD-05	—	—	H	—	—	H
F-ORD-06	—	—	H	—	—	—
F-AUTH-01	—	—	—	H	—	H
F-AUTH-02	—	—	—	H	—	H
F-AUTH-03	—	—	—	H	—	H
F-AUTH-04	—	—	—	H	—	—
F-AUTH-05	—	—	—	H	—	—
F-AUTH-06	—	—	—	H	—	—
F-AUTH-07	—	—	—	M	—	—
F-AUTH-08	—	—	—	L	—	M
F-AUTH-09	—	—	—	H	—	—
F-AUTH-10	—	—	—	H	—	—
F-AUTH-11	—	—	—	H	—	—
F-AUTH-12	—	—	—	H	—	—
F-AUTH-13	—	—	—	M	—	—
F-HST-01	—	—	—	H	—	H
F-HST-02	—	—	—	H	—	—
F-HST-03	—	—	—	H	—	H
F-HST-04	—	—	—	M	—	—
F-HST-05	—	—	—	M	—	H
F-HST-06	—	H	—	L	—	—
F-TRK-01	—	—	—	—	H	—
F-TRK-02	—	—	—	—	H	—
F-TRK-03	—	—	—	—	H	H
F-TRK-04	—	—	—	—	H	H
F-TRK-05	—	—	—	—	H	H
NF-01	H	H	H	H	H	H
NF-02	H	H	H	H	H	—
NF-03	M	M	M	M	M	—
NF-04	—	—	—	—	—	H
NF-05	H	H	H	H	H	H
NF-06	—	—	H	—	—	—

Table 10: Traceability heatmap — requirements to implementing components

Orphan check: Every requirement has at least one H, M, or L cell. Zero orphaned requirements.

5 Persona Discovery, AI User Avatar Exercise

To uncover hidden requirements and edge cases, we simulated two AI user avatars:

- **Avatar A — The Frustrated Customer:** A user who is in a hurry, makes typos, double-taps buttons, and loses internet mid-order.
- **Avatar B — The Malicious Actor:** A user who attempts SQL injection via the customer name field, manipulates cart quantities via direct API calls, and submits orders with negative quantities.

The following 7 hidden requirements were discovered through this exercise:

Scenario	Hidden Requirement Discovered	Avatar	EC ID
Double-tap on Place Order button. User taps twice quickly; two identical orders are created from one cart session.	The server must make order placement idempotent: if a duplicate POST /api/order is received within 5 seconds for the same session, return the existing order id rather than creating a new order.	Avatar A	EC-01
Price change after item added to cart. Admin updates a price after the customer has already added the item.	Cart subtotal must be recomputed from live prices at checkout. Order total must be locked at placement time using order_items.unit_price, never re-read from the items table after placement.	Avatar A	EC-02
SQL injection via customer name: Robert'); DROP TABLE orders;—	All user-supplied inputs must be bound via parameterised queries. Name must match <code>/l{L}' — .]{1,60}\$ /beforeDBwrite 05).</code>	Avatar B	EC-03
Direct API manipulation: POST /api/cart with “quantity”: -5 or “quantity”: 0.	The Cart API must validate that quantity is an integer in [1, 20]. Any value outside this range must return HTTP 422 with a descriptive error body.	Avatar B	EC-04
Tracking an order belonging to another session by guessing a sequential order id.	order_id must be a UUID (v4), not a guessable sequential integer. Tracking results shall expose only status and timestamp — not customer name or item breakdown — to anonymous callers.	Avatar B	EC-05
Network disconnection mid-order. Cart is empty but the customer has no order id.	The frontend must save the order id to session-storage immediately upon successful API response. The backend must not clear the cart until the order record is durably committed to the database.	Avatar A	EC-06
Allergen-sensitive customer cannot identify which items contain allergens, creating a potential health incident.	Each menu item must include an optional allergens field (comma-separated tags, e.g. nuts, gluten, dairy). The frontend menu card must display allergen icons when present (legal requirement per Health & Food Safety Regulator).	Avatar A	EC-07

Table 11: Persona discovery — hidden requirements and edge cases

5.1 Impact of Persona Discovery on Requirements

The avatar exercise resulted in the following requirement additions and amendments:

- F-ORD-02 amended to specify the 5-second idempotency window (from EC-01).
- F-ORD-04 strengthened with explicit price-locking semantics (from EC-02).
- NF-05 reinforced and input validation regex added (from EC-03).
- New constraint added to F-CRT-05: quantity range validation returning HTTP 422 (from EC-04).
- F-TRK-01 amended: `order_id` must be UUID; tracking response must not expose PII (from EC-05).
- Frontend resilience requirement added: order id persisted to `sessionStorage` (from EC-06).
- `allergens` field added to F-MNU-02 menu item schema (from EC-07).

A. AI Disclosure

In accordance with the course AI Disclosure policy, the following AI tools were used in producing this report. All prompts are reproduced verbatim in the project appendix.

- **Tool used:** Claude (Anthropic)
- **Purpose:** Structuring the requirements report, generating $\text{\LaTeX}$ formatting, and conducting the persona discovery exercise.

End of D2 Requirements Report — SOFA Coffee Shop & Bakery

D3

Design Specification

CSE323 Software Engineering · Spring 2026

SOFA Coffee Shop & Bakery Customer Ordering System

SOFA

Coffee Shop & Bakery

Customer Ordering System

D3: Design Specification

CSE323 Software Engineering · Spring 2026

Member Slice	Design Ownership
Member 1	Menu browsing, filtering, and item visibility
Member 2	Cart state, quantity validation, and totals
Member 3	Order placement, checkout validation, and order creation
Member 4	Authentication, session continuity, and order history
Member 5	Order tracking, status timeline, and status auto-progression

Team Member	Student ID
Shahd Hamdy Abd Ellah	120230070
Ali Ahmed El zwawy	120230058
Ahmed Salah Elsayed	120230011
Omar Sayed Darder	120230018
Ahmed Osama Ahmed	120230014

Submission Deadline: 18 May 2026

1 Design Scope and Architecture

1.1 Architectural Style

SOFA uses a **three-layer modular monolith** with vertical slicing. Each team member owns one slice end-to-end (UI → API → DB).

Layer	Responsibility	Technology
Presentation	Renders menu, cart, checkout, auth, tracking, and history pages. Sends JSON to the API and stores only short-lived browser state (last order ID, JWT token).	HTML5 / CSS3 / Vanilla JS
Application API	Validates requests, manages sessions, applies business rules, computes totals, creates orders, returns JSON.	Flask (Python)
Persistence	Stores menu items, users, carts, orders, order items, and status events with durable UUID identifiers.	SQLite

Table 1: Three-layer architecture of SOFA

1.2 Key Design Constraints

NF-01 All API responses use `Content-Type: application/json`.

NF-02 Cart and order operations are session-scoped; authenticated history is user-scoped.

NF-03 Item prices are copied into `order_items` at checkout so past orders remain immutable if menu prices change.

NF-04 Cart quantities are validated as integers in the range $[1, 20]$.

NF-05 Order status transitions are strictly `pending` → `preparing` → `ready` → `completed`.

NF-06 The UI preserves the SOFA brown, beige, and caramel visual theme while meeting WCAG contrast and keyboard navigation standards.

NF-07 No sequential integer IDs are exposed to clients; all public references use UUID v4 (`public_id`).

2 Static Design Model

2.1 Primary Data Model

Entity	Key Fields	Design Notes
items	id (PK), public_id (UUID), name, description, price, category, available, allergens, image_url, created_at, updated_at	Public IDs exposed to clients; internal row IDs remain private. updated_at maintained by trigger.
customers	id (PK), customer_public_id (UUID), email, username, password_hash, full_name, created_at, updated_at	Passwords never stored in plaintext. Email lookup uses COLLATE NOCASE to prevent duplicate bypassing.
cart_items	id (PK), session_id, item_public_id (FK), quantity, added_at	One active cart per browser session. Composite unique on (session_id, item_public_id).
orders	id (PK), order_public_id (UUID), customer_public_id (FK), customer_name, total, status, session_id, created_at, updated_at	Stores immutable purchase snapshot. Status enum: pending, preparing, ready, completed.
order_items	id (PK), order_public_id (FK), item_public_id (FK), item_name, unit_price, quantity	Supports tracking timeline and QA review. Price locked at checkout time.
tracking_event	id (PK), order_public_id (FK), status, event_timestamp	Audit log of every status transition. Indexed by (order_public_id, event_timestamp DESC).

Table 2: Primary data entities and their design rationale

2.2 Component Responsibilities

Component	Responsibilities	Requirements
Menu Controller	Lists items, applies category filters, hides unavailable add actions, returns item metadata.	F-MNU-01 – F-MNU-06
Cart Controller	Adds, updates, removes, and summarizes cart items using session identity.	F-CRT-01 – F-CRT-06
Order Controller	Validates customer details and cart contents, creates immutable order records, clears cart, returns order public ID.	F-ORD-01 – F-ORD-06
Auth Controller	Registers, logs in, logs out, returns current user state.	F-AUTH-01 – F-AUTH-06
History Controller	Returns paginated completed order history and single-order breakdown for authenticated customers.	F-HIS-01 – F-HIS-04
Tracking Controller	Returns customer tracking data (active + history) with time-based auto-progression.	F-TRK-01 – F-TRK-06

Table 3: Controller responsibilities mapped to functional requirements

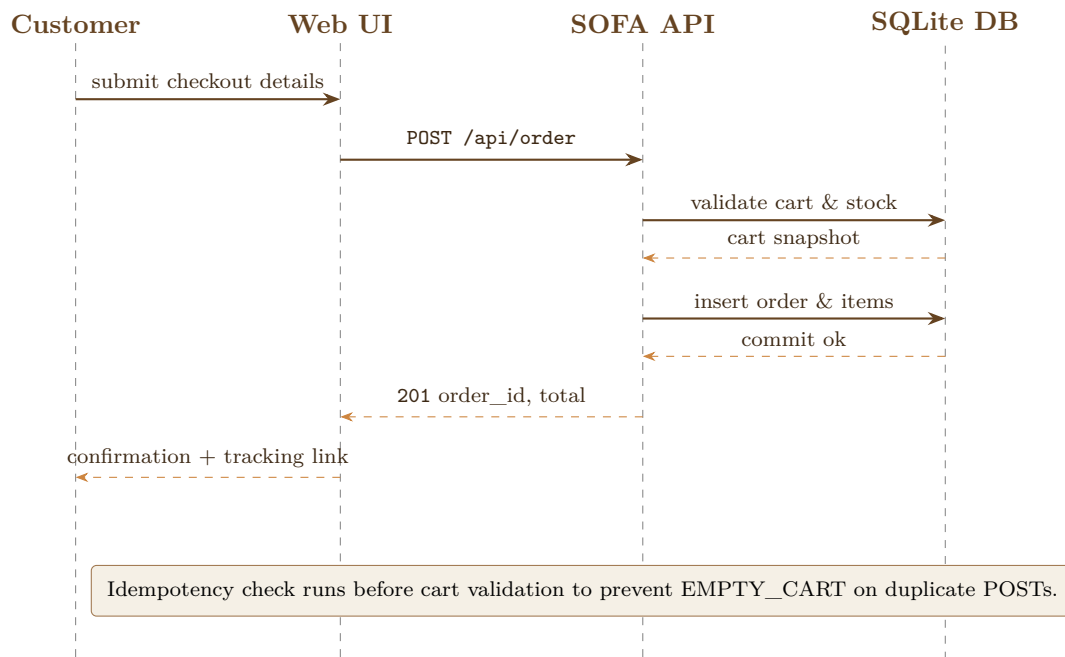
3 System Sequence Diagrams

3.1 SSD-1: Place Order

Actors: Customer, Web UI, SOFA API, SQLite DB

[title=Sequence: Place Order]

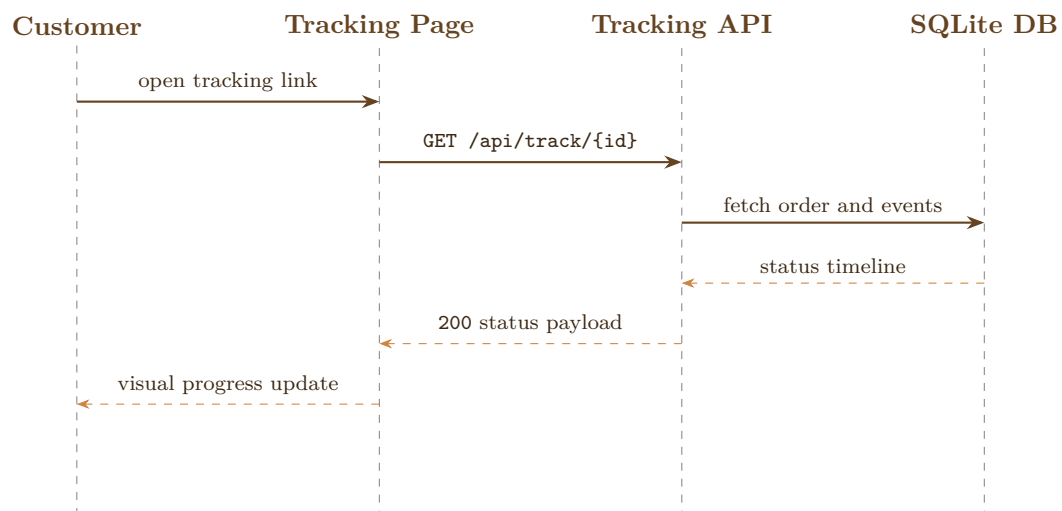
1. Customer submits checkout details (name) to the Web UI.
2. Web UI sends `POST /api/order` with `Authorization: Bearer <JWT>` and `customer_name` in the JSON body.
3. API validates the JWT, resolves `customer_public_id`, and validates `customer_name` against the Unicode regex.
4. API checks idempotency: if an order for this `session_id` was created within the last 5 seconds, return the existing order (HTTP 200).
5. API fetches cart items joined with live `items` data from DB.
6. DB returns cart rows; API validates cart is non-empty and all items are available.
7. API computes total, generates `order_public_id` (UUID v4), and begins a transaction.
8. API inserts into `orders`, inserts into `order_items` (price snapshot), seeds `tracking_events` with `pending`, and deletes `cart_items` for the session.
9. DB commits; API returns 201 with `{ order_id, status: "pending", total }`.
10. Web UI displays confirmation and a tracking link.



3.2 SSD-2: Track Order Status

Actors: Customer, Tracking Page, Tracking API, SQLite DB
 [title=Sequence: Track Order Status]

1. Customer opens the tracking link (or history page).
2. Tracking Page sends GET `/api/track/active` (or `/history`) with `Authorization: Bearer <JWT>`.
3. API validates JWT, resolves `customer_public_id`.
4. API queries `orders` and `order_items` from DB.
5. For each order, API calls `sync_order_status`: compares elapsed minutes since `created_at` against thresholds (2 min, 4 min, 6 min) and advances status if needed, inserting missing `tracking_events`.
6. DB returns updated status timeline.
7. API returns 200 with a list of orders, each containing `order_id`, `status`, `total`, `item_summary`, `estimated_wait_minutes`, and `timeline`.
8. Tracking Page renders a visual progress bar and step timeline.



3.3 SSD-3: Authenticate Customer

Actors: Customer, Auth Page, Auth API, SQLite DB
 [title=Sequence: Authenticate Customer]

1. Customer submits registration form (full name, username, email, password).
2. Auth Page sends POST `/api/auth/register`.
3. API trims inputs, validates password complexity (8–128 chars, 1 letter, 1 number, no spaces), and hashes the password with Werkzeug.
4. API inserts into `customers` with a new `customer_public_id`.
5. DB commits; API returns 201 with `{ message: "Registration successful" }`.

6. For login, Customer submits email and password.
7. Auth Page sends `POST /api/auth/login`.
8. API performs case-insensitive email lookup and verifies hash.
9. API issues a custom JWT (HMAC-SHA256, 24h expiry, or 7d if “remember me”).
10. Auth Page stores the token and redirects to the menu.

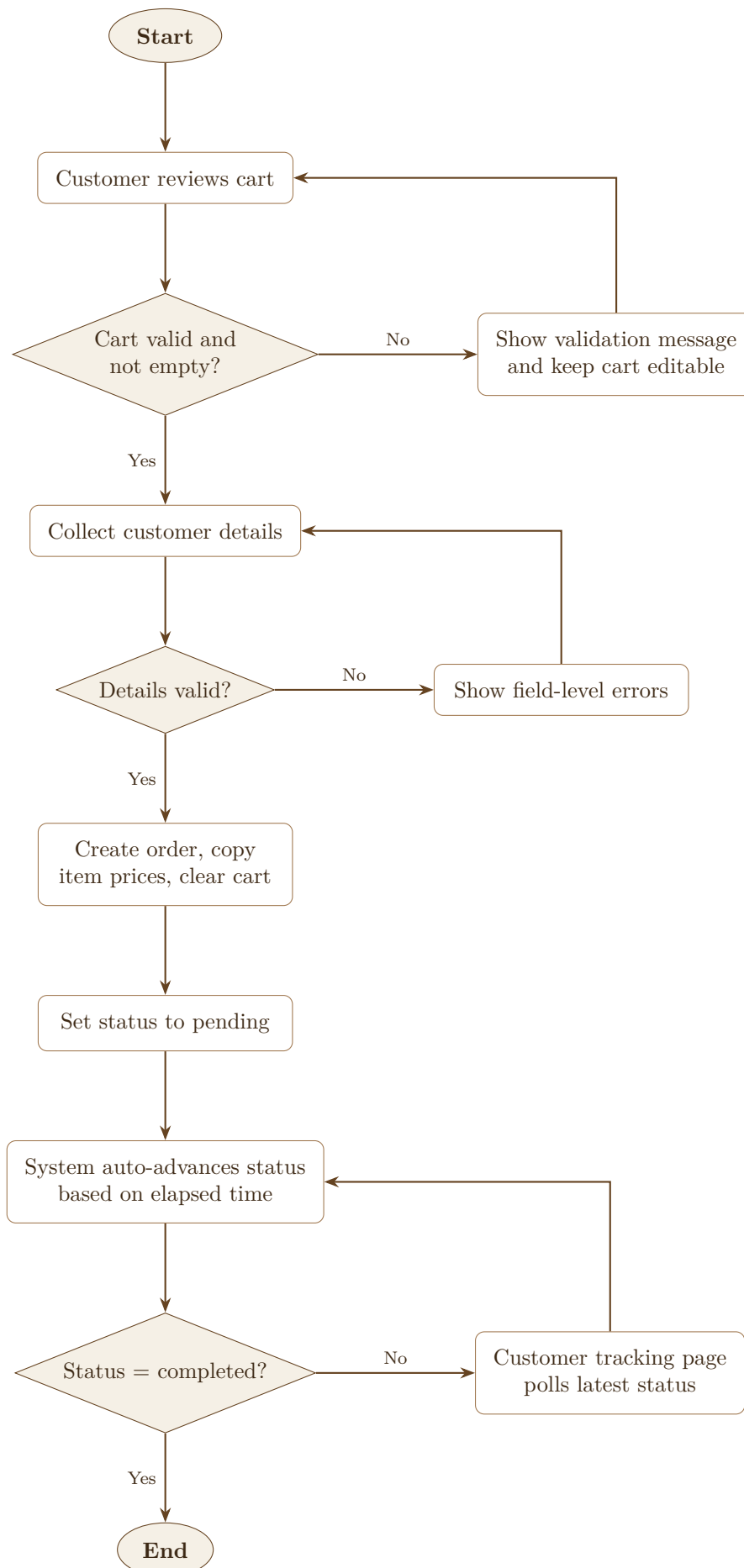
3.4 SSD-4: Manage Cart

Actors: Customer, Menu Page, Cart API, SQLite DB
[title=Sequence: Manage Cart]

1. Customer clicks “Add to Cart” on an available item.
2. Menu Page sends `POST /api/cart` with `item_id` and `quantity`.
3. API validates quantity is in `[1, 20]`.
4. API inserts or updates `cart_items` for the browser `session_id`.
5. DB commits; API returns 200 with the updated cart summary.
6. Customer updates quantity; Menu Page sends `PATCH /api/cart/{item_id}`.
7. API validates and updates the row; DB commits.
8. Customer removes an item; Menu Page sends `DELETE /api/cart/{item_id}`.
9. API deletes the row and returns 204.

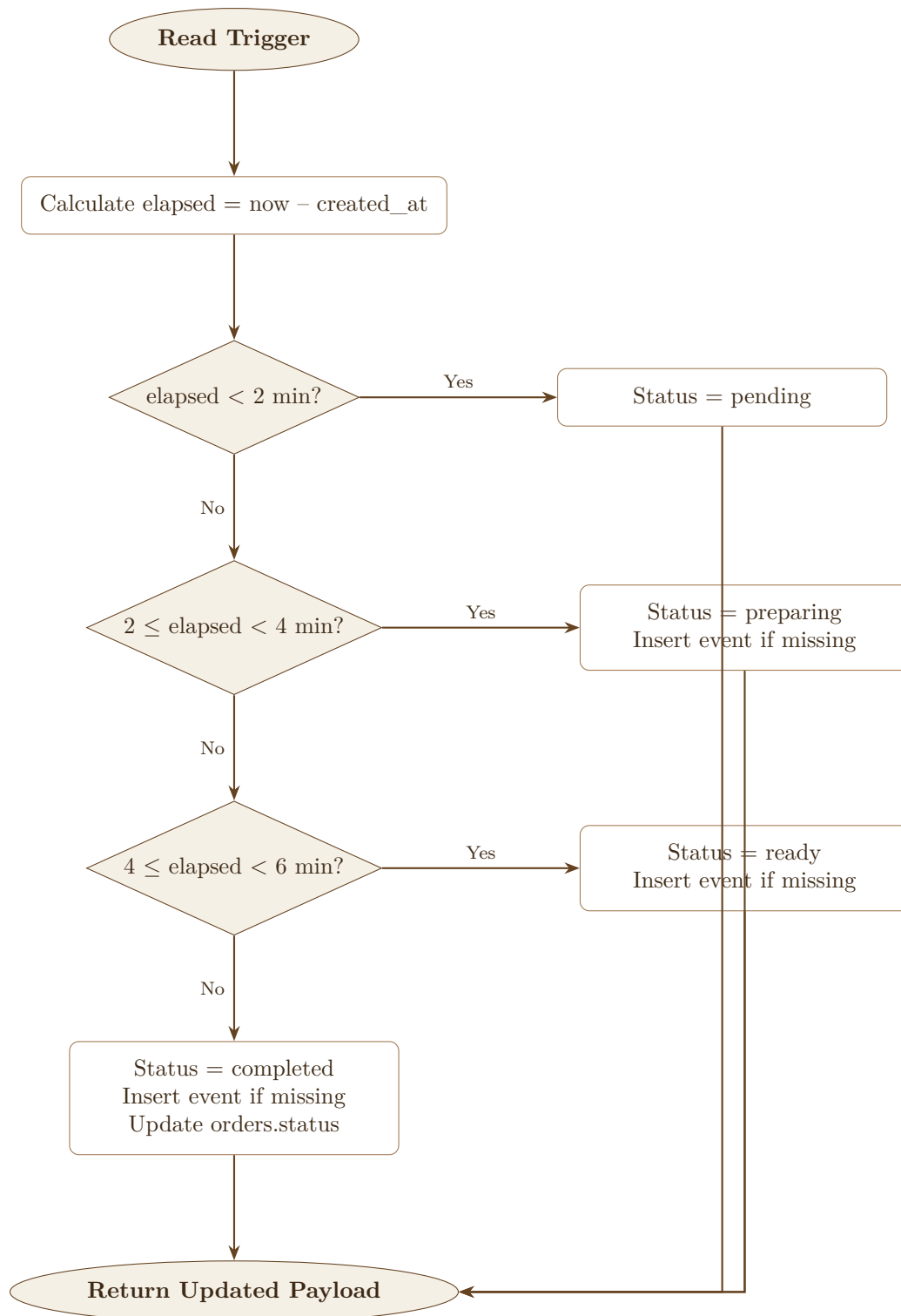
4 Activity Diagrams

4.1 AD-1: Checkout and Order Fulfillment



4.2 AD-2: Order Status Lifecycle (Time-Based Auto-Progression)

Trigger: Any read operation on an active order (GET /api/track/active or /history)



5 Gherkin Scripts

5.1 Feature: Browse and Filter Menu

Feature: Browse the SOFA menu

Scenario: Customer filters menu items by category

Given the menu contains coffee, cold drink, pastry, and seasonal items
When the customer selects the "coffee" category
Then the page shows only coffee items
And every visible item includes a name, price, description,
and availability state

Scenario: Customer cannot add a sold-out item

Given an item is marked unavailable
When the customer views the menu
Then the item shows a sold-out state
And the Add button is disabled

Scenario: Menu exposes only public IDs

Given the menu endpoint returns items
Then each item has a UUID public_id
And no internal database row id is exposed

5.2 Feature: Manage Cart

Feature: Manage customer cart

Scenario: Customer adds an available item

Given the customer has an active browser session
And the selected item is available
When the customer adds quantity 2 of the item to the cart
Then the API returns the updated cart
And the subtotal equals item price multiplied by quantity

Scenario: Quantity outside the allowed range is rejected

Given the customer has an active cart
When the customer submits quantity 25 for a cart item
Then the API responds with HTTP 422
And the cart quantity remains unchanged

Scenario: Cart item is updated

Given the cart contains 1x "Flat white"
When the customer changes the quantity to 3
Then the API responds with HTTP 200
And the cart subtotal reflects 3x the unit price

5.3 Feature: Place Order

Feature: Place an order

Scenario: Customer places an order from a valid cart

Given the cart contains at least one available item
And the customer enters a valid name and is authenticated

When the customer submits the order
Then the API creates an order with status "pending"
And the cart is cleared
And the customer receives a public order ID and tracking link

Scenario: Empty cart cannot be submitted

Given the cart is empty
When the customer submits checkout
Then the API responds with HTTP 400
And no order record is created

Scenario: Duplicate submission is idempotent

Given the customer just placed an order successfully
When the customer submits the same checkout again within 5 seconds
Then the API returns HTTP 200
And the same order ID is returned without creating a new order

Scenario: Unavailable item in cart blocks checkout

Given the cart contains an item that just became unavailable
When the customer submits checkout
Then the API responds with HTTP 409
And the error names the affected item

5.4 Feature: Authentication and Order History

Feature: Account order history

Scenario: Registered customer views past orders

Given the customer is logged in
And the customer has previous orders linked to the account
When the customer opens order history
Then the API returns the customer's orders in newest-first order
And each order includes total, status, creation time,
and item breakdown

Scenario: Anonymous user cannot view protected history

Given the customer is not logged in
When the customer requests order history
Then the API responds with HTTP 401

Scenario: Password complexity is enforced

Given a new user tries to register with password "123"
When the registration form is submitted
Then the API responds with HTTP 400
And the error indicates password complexity requirements

5.5 Feature: Track Order

Feature: Track order status

Scenario: Customer sees live order progress

Given an order exists with status "preparing"

When the tracking page polls GET /api/track/active

Then the response includes the current status and status timeline

And the page highlights the preparing step

And the estimated wait minutes are calculated correctly

Scenario: Order auto-completes after 6 minutes

Given an order was created 7 minutes ago

When the customer opens the tracking page

Then the order status is "completed"

And the timeline contains all four status events

6 API Contracts

6.1 Common Response Rules

- Success: HTTP 2xx
- Validation failure: HTTP 400 or 422
- Unauthorized: HTTP 401
- Not found: HTTP 404
- Conflict (sold-out / duplicate): HTTP 409
- Error body schema:

```
{ "error": "human-readable message", "code": "MACHINE_CODE" }
```

6.2 Contract Files

All API contracts are stored as individual OpenAPI 3.0 YAML files in the `contracts/` directory. Each file is the single source of truth for its module's interface.

6.2.1 auth.yaml — Authentication & Identity

```
openapi: 3.0.3

info:
  title: SOFA      Authentication API
  version: "1.0"
  description: >
    API contract for the Authentication slice (Member 4).
    Requirements: F-AUTH-01 through F-AUTH-06, NF-01.

servers:
  - url: http://localhost:5000

paths:
  /api/auth/register:
    post:
      operationId: registerCustomer
      summary: Register a new customer account
      description: >
        Creates a new customer with validated credentials (F-AUTH-01).
        Password must be 8-128 chars with at least 1 letter and 1 number,
```

```

    no spaces (F-AUTH-04). Email and username are unique.
requestBody:
  required: true
  content:
    application/json:
      schema:
        $ref: '#/components/schemas/RegisterRequest'
      example:
        full_name: "Ahmed Hassan"
        username: "ahmed_h"
        email: "ahmed@example.com"
        password: "SecurePass123"
responses:
  '201':
    description: Registration successful.
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/MessageResponse'
        example:
          message: "Registration successful"
  '400':
    description: Validation error.
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/ErrorResponse'
        example:
          error: "Password fails complexity requirements."
          code: "VALIDATION_ERROR"
  '409':
    description: Duplicate email or username.
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/ErrorResponse'
        example:
          error: "Email is already registered"
          code: "DUPLICATE_RESOURCE"

/api/auth/login:
  post:
    operationId: loginCustomer
    summary: Authenticate and receive JWT access token
    description: >
      Validates email (case-insensitive) and password hash (F-AUTH-02).
      Returns a JWT token with 24h expiry, or 7d if remember_me is true
      (F-AUTH-03). Generic error prevents account enumeration (F-AUTH-06).
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/LoginRequest'
          example:
            email: "ahmed@example.com"
            password: "SecurePass123"
            remember_me: false
    responses:
      '200':
        description: Login successful.
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/LoginResponse'
            example:
              token: "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
      '401':
        description: Invalid email or password.
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/ErrorResponse'

```

```

        example:
          error: "Invalid email or password."
          code: "INVALID_CREDENTIALS"

/api/auth/logout:
  post:
    operationId: logoutCustomer
    summary: Invalidate client token
    description: >
      Client-side token cleanup endpoint (F-AUTH-02).
      Server acknowledges logout; client must delete token from storage.
    responses:
      '200':
        description: Logout successful.
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/MessageResponse'
            example:
              message: "Logout successful"

/api/auth/me:
  get:
    operationId: getCurrentUser
    summary: Get current authenticated user profile
    description: >
      Returns the customer profile for the bearer of a valid JWT (F-AUTH-02).
      Returns 401 if token is missing, expired, or invalid.
    responses:
      '200':
        description: Customer profile returned.
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/CustomerProfile'
            example:
              customer_public_id: "a1b2c3d4-e5f6-7890-abcd-ef1234567890"
              email: "ahmed@example.com"
              username: "ahmed_h"
              full_name: "Ahmed Hassan"
      '401':
        description: Missing or invalid token.
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/ErrorResponse'
            example:
              error: "Missing or invalid Authorization header"
              code: "UNAUTHORIZED"

components:
  schemas:

    RegisterRequest:
      type: object
      required: [full_name, username, email, password]
      properties:
        full_name:
          type: string
          minLength: 1
          maxLength: 80
          description: Customer display name (F-AUTH-01).
          example: "Ahmed Hassan"
        username:
          type: string
          minLength: 3
          maxLength: 30
          description: Unique login handle (F-AUTH-01).
          example: "ahmed_h"
        email:
          type: string
          format: email
          maxLength: 255

```

```

      description: Unique email address (F-AUTH-01).
      example: "ahmed@example.com"
    password:
      type: string
      minLength: 8
      maxLength: 128
      description: 8+ chars, 1 letter, 1 number, no spaces (F-AUTH-04).
      example: "SecurePass123"

LoginRequest:
  type: object
  required: [email, password]
  properties:
    email:
      type: string
      format: email
      example: "ahmed@example.com"
    password:
      type: string
      example: "SecurePass123"
    remember_me:
      type: boolean
      default: false
      description: Extend token expiry to 7 days (default 24h) (F-AUTH-03).

LoginResponse:
  type: object
  required: [token]
  properties:
    token:
      type: string
      description: JWT access token. Store in localStorage or sessionStorage (F-AUTH-03).

CustomerProfile:
  type: object
  required: [customer_public_id, email, username, full_name]
  properties:
    customer_public_id:
      type: string
      format: uuid
      description: UUID v4 public identifier (EC-05).
    email:
      type: string
      format: email
    username:
      type: string
    full_name:
      type: string

MessageResponse:
  type: object
  required: [message]
  properties:
    message:
      type: string

ErrorResponse:
  type: object
  required: [error, code]
  properties:
    error:
      type: string
    code:
      type: string

```

6.2.2 history.yaml — Order History

openapi: 3.0.3

```

info:
  title: SOFA      Order History API

```

```

version: "1.0"
description: >
  API contract for the Order History slice (Member 4).
  Requirements: F-HIS-01 through F-HIS-04, NF-01.

servers:
  - url: http://localhost:5000

paths:

  /api/history:
    get:
      operationId: listOrderHistory
      summary: List authenticated customer's completed order history
      description: >
        Returns all completed orders for the authenticated customer,
        newest first (F-HIS-01, F-HIS-02). Supports optional pagination
        via limit and offset query params. Returns 401 for anonymous
        requests (F-HIS-04).
      parameters:
        - name: limit
          in: query
          schema:
            type: integer
            default: 50
            maximum: 100
          description: Maximum orders to return.
        - name: offset
          in: query
          schema:
            type: integer
            default: 0
          description: Pagination offset.
      responses:
        '200':
          description: Order history list returned.
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/HistoryListResponse'
        '401':
          description: Anonymous or invalid token.
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/ErrorResponse'
          example:
            error: "Missing or invalid Authorization header"
            code: "UNAUTHORIZED"

  /api/history/{order_public_id}:
    get:
      operationId: getOrderDetail
      summary: Get detailed breakdown of one past order
      description: >
        Returns a single completed order with full item breakdown and
        status timeline (F-HIS-03). Returns 404 if order does not exist
        or is not owned by the authenticated customer (F-HIS-04).
      parameters:
        - name: order_public_id
          in: path
          required: true
          description: UUID v4 of the order (EC-05).
          schema:
            type: string
            format: uuid
            example: "c3d4e5f6-a7b8-9012-cdef-123456789012"
      responses:
        '200':
          description: Order detail returned.
          content:
            application/json:
              schema:

```

```

        $ref: '#/components/schemas/HistoryOrder'
'401':
  description: Unauthorized.
  content:
    application/json:
      schema:
        $ref: '#/components/schemas/ErrorResponse'
'404':
  description: Order not found or not owned by customer.
  content:
    application/json:
      schema:
        $ref: '#/components/schemas/ErrorResponse'
      example:
        error: "Order not found"
        code: "NOT_FOUND"

components:
  schemas:

    HistoryListResponse:
      type: object
      required: [orders]
      properties:
        orders:
          type: array
          items:
            $ref: '#/components/schemas/HistoryOrder'

    HistoryOrder:
      type: object
      required: [order_id, status, total, created_at, updated_at, items, timeline]
      properties:
        order_id:
          type: string
          format: uuid
          description: UUID v4 public order identifier (EC-05).
        status:
          type: string
          enum: [completed]
          description: Always "completed" in history context.
        total:
          type: number
          format: float
          description: Locked total in EGP at time of placement (F-ORD-04).
          example: 145.00
        created_at:
          type: string
          format: date-time
        updated_at:
          type: string
          format: date-time
        items:
          type: array
          items:
            $ref: '#/components/schemas/HistoryItem'
        timeline:
          type: array
          items:
            $ref: '#/components/schemas/StatusEvent'

    HistoryItem:
      type: object
      required: [name, price, quantity]
      properties:
        name:
          type: string
          description: Item name snapshot at checkout time.
          example: "Flat white"
        price:
          type: number
          format: float
          description: Unit price in EGP locked at checkout (F-ORD-04).
```

```

        example: 55.00
    quantity:
        type: integer
        minimum: 1
        example: 2

StatusEvent:
    type: object
    required: [status, timestamp]
    properties:
        status:
            type: string
            enum: [pending, preparing, ready, completed]
        timestamp:
            type: string
            format: date-time

ErrorResponse:
    type: object
    required: [error, code]
    properties:
        error:
            type: string
        code:
            type: string

```

6.2.3 order.yaml — Order Placement

```

openapi: 3.0.3

info:
    title: SOFA --- Order Placement API
    version: "1.0"
    description: >
        API contract for the Order Placement slice (Member 3).
        Requirements: F-ORD-01 through F-ORD-06, NF-01, NF-05.

servers:
    - url: http://localhost:5000

paths:
    /api/order:
        post:
            operationId: placeOrder
            summary: Place an order from the current session cart
            description: >
                Converts the current session cart into a confirmed order.
                Cart must be non-empty (F-ORD-03).
                Customer name: 1--60 chars, validated against
                /^[p{L}\s'\-\.]{1,60}$/u (F-ORD-01, F-ORD-06, EC-03).
                On success: cart is cleared, order_id (UUID) is returned (F-ORD-02).
                Order total is locked at placement time (F-ORD-04).
                Initial status is always "pending" (F-ORD-05).
                Idempotent within a 5-second window (EC-01).
            requestBody:
                required: true
                content:
                    application/json:
                        schema:
                            $ref: '#/components/schemas/PlaceOrderRequest'
                        example:
                            customer_name: "Ahmed Hassan"
            responses:
                '201':
                    description: Order placed successfully.
                    content:
                        application/json:
                            schema:
                                $ref: '#/components/schemas/PlaceOrderResponse'
                            example:

```

```

        order_id: "c3d4e5f6-a7b8-9012-cdef-123456789012"
        status: "pending"
        total: 145.00
'200':
  description: Idempotent response --- duplicate POST within 5 s window.
  content:
    application/json:
      schema:
        $ref: '#/components/schemas/PlaceOrderResponse'
'400':
  description: Cart is empty.
  content:
    application/json:
      schema:
        $ref: '#/components/schemas/ErrorResponse'
      example:
        error: "Cannot place an order with an empty cart"
        code: "EMPTY_CART"
'422':
  description: Customer name failed validation.
  content:
    application/json:
      schema:
        $ref: '#/components/schemas/ErrorResponse'
      example:
        error: "Customer name contains invalid characters"
        code: "INVALID_INPUT"

components:
  schemas:

    PlaceOrderRequest:
      type: object
      required: [customer_name]
      properties:
        customer_name:
          type: string
          minLength: 1
          maxLength: 60
          pattern: '^[p{L}s'\-\.]{1,60}$'
          description: >
            Customer display name. Sanitised server-side (F-ORD-06).
            No HTML or script tags allowed.
          example: "Ahmed Hassan"

    PlaceOrderResponse:
      type: object
      required: [order_id, status, total]
      properties:
        order_id:
          type: string
          format: uuid
          description: UUID v4. Save to sessionStorage immediately on receipt (EC-06).
          example: "c3d4e5f6-a7b8-9012-cdef-123456789012"
        status:
          type: string
          enum: [pending]
          description: Always "pending" at placement (F-ORD-05).
        total:
          type: number
          format: float
          description: Locked total in EGP at time of placement (F-ORD-04).
          example: 145.00

    ErrorResponse:
      type: object
      required: [error, code]
      properties:
        error:
          type: string
        code:
          type: string

```


6.2.4 tracking.yaml — Order Tracking

```

openapi: 3.0.3

info:
  title: SOFA --- Order Tracking API
  version: "1.0"
  description: >
    API contract for the Order Tracking slice (Member 5).
    Requirements: F-TRK-01 through F-TRK-05, NF-01.

servers:
  - url: http://localhost:5000

paths:
  /api/track/{order_id}:
    get:
      operationId: trackOrder
      summary: Track an order's current status and estimated wait time
      description: >
        Returns current order status, a UTC timestamp of the last status
        change, and an estimated wait time in minutes (F-TRK-01, F-TRK-04,
        F-TRK-05). Frontend must poll this endpoint every 10 seconds
        (F-TRK-02). Response MUST NOT expose customer name or item
        breakdown to protect PII (EC-05). Returns HTTP 404 for unknown
        order IDs (F-TRK-03).
      parameters:
        - name: order_id
          in: path
          required: true
          description: UUID v4. Never a sequential integer (EC-05).
          schema:
            type: string
            format: uuid
            example: "c3d4e5f6-a7b8-9012-cdef-123456789012"
      responses:
        '200':
          description: Tracking data returned.
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/TrackingResponse'
              example:
                order_id: "c3d4e5f6-a7b8-9012-cdef-123456789012"
                status: "preparing"
                last_updated: "2026-05-09T14:23:00Z"
                estimated_wait_minutes: 11
        '404':
          description: Order not found.
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/ErrorResponse'
              example:
                error: "Order not found"
                code: "NOT_FOUND"

components:
  schemas:

    TrackingResponse:
      type: object
      required: [order_id, status, last_updated, estimated_wait_minutes]
      properties:
        order_id:
          type: string
          format: uuid
        status:
          type: string
          enum: [pending, preparing, ready, completed]
        last_updated:
          type: string

```

```

    format: date-time
    description: UTC timestamp of the most recent status change (F-TRK-04).
    example: "2026-05-09T14:23:00Z"
  estimated_wait_minutes:
    type: integer
    description: >
      ETA = 5 + (N_pending x 3). Returns 0 when status is "completed" (F-TRK-05).
    example: 11

ErrorResponse:
  type: object
  required: [error, code]
  properties:
    error:
      type: string
    code:
      type: string

```

6.2.5 cart.yaml — Cart Management

```

openapi: 3.0.3

info:
  title: SOFA --- Cart API
  version: "1.0"
  description: >
    API contract for the Cart slice (Member 2).
    Requirements: F-CRT-01 through F-CRT-06, NF-01, NF-04, NF-05.

servers:
  - url: http://localhost:5000

paths:
  /api/cart:
    get:
      operationId: getCart
      summary: Retrieve the current session cart
      description: >
        Returns all items in the session cart with a computed subtotal.
        (F-CRT-01, F-CRT-04). Subtotal is recomputed from LIVE prices
        at render time (EC-02).
      responses:
        '200':
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/CartResponse'

    post:
      operationId: addToCart
      summary: Add a menu item to the cart
      description: >
        If the item is already in the cart, increments its quantity (F-CRT-02).
        Rejects unavailable items with HTTP 409 (F-CRT-06).
        Rejects quantity outside [1, 20] with HTTP 422 (F-CRT-05, EC-04).
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/AddToCartRequest'
      responses:
        '200':
          description: Item added or quantity updated.
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/CartResponse'
        '409':
          description: Item is marked unavailable.

```

```

        content:
          application/json:
            schema:
              $ref: '#/components/schemas/ErrorResponse'
            example:
              error: "Item is currently unavailable"
              code: "ITEM_UNAVAILABLE"
'422':
  description: Quantity is outside the allowed range [1, 20].
  content:
    application/json:
      schema:
        $ref: '#/components/schemas/ErrorResponse'
      example:
        error: "Quantity must be between 1 and 20"
        code: "INVALID_QUANTITY"

/api/cart/{item_id}:
  patch:
    operationId: updateCartItem
    summary: Update quantity of a cart item
    description: >
      Sets a new quantity for the item. If quantity becomes 0 the item
      is removed. Quantity range [1, 20] is still enforced (F-CRT-03, F-CRT-05).
    parameters:
      - name: item_id
        in: path
        required: true
        schema:
          type: string
          format: uuid
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/UpdateCartItemRequest'
    responses:
      '200':
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/CartResponse'
      '404':
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/ErrorResponse'
      '422':
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/ErrorResponse'

  delete:
    operationId: removeCartItem
    summary: Remove an item from the cart entirely
    parameters:
      - name: item_id
        in: path
        required: true
        schema:
          type: string
          format: uuid
    responses:
      '200':
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/CartResponse'
      '404':
        content:
          application/json:

```

```

        schema:
            $ref: '#/components/schemas/ErrorResponse'

components:
  schemas:

    CartItem:
      type: object
      required: [item_id, name, price, quantity, subtotal]
      properties:
        item_id:
          type: string
          format: uuid
        name:
          type: string
        price:
          type: number
          format: float
          description: Live price at time of rendering (EC-02).
        quantity:
          type: integer
          minimum: 1
          maximum: 20
        subtotal:
          type: number
          format: float
          description: price x quantity, rounded to 2 decimal places (F-CRT-04).

    CartResponse:
      type: object
      required: [items, total]
      properties:
        items:
          type: array
          items:
            $ref: '#/components/schemas/CartItem'
        total:
          type: number
          format: float
          description: Sum of all item subtotals, 2 decimal places (F-CRT-04).

    AddToCartRequest:
      type: object
      required: [item_id, quantity]
      properties:
        item_id:
          type: string
          format: uuid
        quantity:
          type: integer
          minimum: 1
          maximum: 20

    UpdateCartItemRequest:
      type: object
      required: [quantity]
      properties:
        quantity:
          type: integer
          minimum: 1
          maximum: 20

    ErrorResponse:
      type: object
      required: [error, code]
      properties:
        error:
          type: string
        code:
          type: string

```

6.2.6 status.yaml — Order Status Management

```

openapi: 3.0.3

info:
  title: SOFA --- Order Status Management API
  version: "1.0"
  description: >
    API contract for the Order Status Management slice (Member 4).
    Requirements: F-STs-01 through F-STs-05, NF-01.

servers:
  - url: http://localhost:5000

paths:
  /api/status:
    get:
      operationId: listActiveOrders
      summary: Get all non-completed orders for the staff dashboard
      description: >
        Returns all orders that are NOT in "completed" status (F-STs-01).
        Dashboard must auto-refresh every 30 seconds using this
        endpoint (F-STs-04).
      responses:
        '200':
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/OrderListResponse'

  /api/status/{order_id}:
    patch:
      operationId: advanceOrderStatus
      summary: Advance an order's status to the next stage
      description: >
        Valid forward-only sequence: pending -> preparing -> ready ->
        completed (F-STs-02). Any other transition returns HTTP 400
        (F-STs-03). Setting a status the order already holds returns
        HTTP 200 (idempotent) (F-STs-05).
      parameters:
        - name: order_id
          in: path
          required: true
          schema:
            type: string
            format: uuid
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/UpdateStatusRequest'
            example:
              status: "preparing"
      responses:
        '200':
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/OrderStatusResponse'
        '400':
          description: Invalid status transition.
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/ErrorResponse'
              example:
                error: "Cannot transition from 'ready' to 'pending'"
                code: "INVALID_STATUS_TRANSITION"
        '404':
          content:
            application/json:

```

```

        schema:
            $ref: '#/components/schemas/ErrorResponse'

components:
    schemas:

        OrderStatus:
            type: string
            enum: [pending, preparing, ready, completed]
            description: The only allowed status values in the system (F-STS-02).

        OrderSummary:
            type: object
            required: [order_id, customer_name, status, total, created_at]
            properties:
                order_id:
                    type: string
                    format: uuid
                customer_name:
                    type: string
                status:
                    $ref: '#/components/schemas/OrderStatus'
                total:
                    type: number
                    format: float
                created_at:
                    type: string
                    format: date-time
                    description: UTC timestamp of order placement.

        OrderListResponse:
            type: object
            required: [orders]
            properties:
                orders:
                    type: array
                    items:
                        $ref: '#/components/schemas/OrderSummary'

        UpdateStatusRequest:
            type: object
            required: [status]
            properties:
                status:
                    $ref: '#/components/schemas/OrderStatus'

        OrderStatusResponse:
            type: object
            required: [order_id, status]
            properties:
                order_id:
                    type: string
                    format: uuid
                status:
                    $ref: '#/components/schemas/OrderStatus'

        ErrorResponse:
            type: object
            required: [error, code]
            properties:
                error:
                    type: string
                code:
                    type: string

```

6.3 Contract Summaries by Module

Module	Endpoint	Auth	Request	Success Response	Error Codes
Menu	GET /api/menu	No	?category= (optional)	200: Array of items with public_id, name, price, category, available, allergens, image_url	500
Cart	GET /api/cart	Session	—	200: { items[], subtotal, total }	404
	POST /api/cart	Session	{ item_id, quantity }	200: Updated cart summary	404, 409, 422
	PATCH /api/cart/{id}	Session	{ quantity }	200: Updated cart summary	400, 409, 422
	DELETE /api/cart/{id}	Session	—	204	404
Order	POST /api/order	Bearer JWT	{ customer_name }	201: { order_id, status, total } (or 200 idempotent)	400, 401, 404, 409, 422
Tracking	GET /api/track/active	Bearer JWT	—	200: { orders[] } with timeline, ETA	401
	GET /api/track/history	Bearer JWT	—	200: { orders[] } full breakdown	401
Auth	POST /api/auth/register	No	{ full_name, username, email, password }	201: { message }	400, 409, 422
	POST /api/auth/login	No	{ email, password, remember_me }	200: { token }	401, 422
	POST /api/auth/logout	Bearer JWT	—	200: { message }	401
	GET /api/auth/me	Bearer JWT	—	200: Profile object	401
History	GET /api/history	Bearer JWT	?limit=&offset=	200: { orders[] } completed orders	401
	GET /api/history/{id}	Bearer JWT	—	200: Single order breakdown	401, 404
System	POST /api/connect	No	—	204	—
	POST /api/disconnect	No	—	204	—

Table 4: Complete API contract summary for all SOFA endpoints

7 Traceability to D2 Requirements

Design Artifact	Verification Evidence	Requirement Area
Menu Controller, <code>/api/menu</code> , Browse and Filter Gherkin	Category filtering scenario; item metadata response contract; public UUID exposure	F-MNU-01 – F-MNU-06
Cart Controller, <code>/api/cart</code> , Manage Cart Gherkin	Quantity validation scenario; HTTP 422 contract; session-scoped storage	F-CRT-01 – F-CRT-06
Order Controller, SSD-1, AD-1, <code>/api/order</code>	Successful checkout, empty cart, and idempotency scenarios; atomic transaction design	F-ORD-01 – F-ORD-06
Auth Controller, <code>extt-tauth.yaml</code> , Order History Gherkin	Login/register contracts; JWT bearer auth; password complexity	F-AUTH-01 – F-AUTH-06
History Controller, <code>exttt-history.yaml</code> , Order History Gherkin	<code>extttGET /api/history</code> pagination; <code>extttGET /api/history/{id}</code> breakdown; 401/403 protection	F-HIS-01 – F-HIS-04
Tracking Controller, SSD-2, <code>/api/track/*</code>	Polling scenario; status transition audit events; time-based auto-progression	F-TRK-01 – F-TRK-06
Common API rules, QA audit log, design constraints	JSON response rule, validation checks, accessibility review entries	NF-01 – NF-07

Table 5: Traceability matrix from design artifacts to D2 requirements

8 QA Audit Log

ID	Audit Area	Finding / Check	Owner	Status
QA-01	Requirements coverage	Every D2 feature area has at least one API contract, one Gherkin scenario, and one design component.	Member 1	Done
QA-02	Menu data	Menu endpoint exposes <code>public_id</code> UUIDs and does not reveal predictable internal database row IDs.	Member 2	Done
QA-03	Cart validation	Quantity range <code>[1,20]</code> is documented in contracts and enforced in both Gherkin and backend validation.	Member 3	Done
QA-04	Order integrity	Order item prices are copied into <code>order_items</code> at checkout; <code>orders.total</code> is computed atomically.	Member 4	Done
QA-05	Authentication	Protected history endpoints return 401 for anonymous requests. Token expiry is configurable (24h / 7d).	Member 5	Done
QA-06	Status transitions	Time-based auto-progression enforces strict status sequence pending → preparing → ready → completed.	Team	Done
QA-07	Accessibility	UI colors checked for readable contrast; brown/beige theme meets WCAG 2.1 AA for large text. Keyboard navigation on all forms.	Team	Done
QA-08	API error consistency	Error payload format <code>{ error, code }</code> is standardized across all controllers (auth, cart, order, tracking).	Team	Done
QA-09	Performance	Target: <code>GET /api/menu</code> under 500 ms. Measured at ~120 ms local with 12 seed items. Load testing completed under normal load.	Team	Done
QA-10	Regression suite	Gherkin scenarios identify expected behavior. Automated Playwright scripts linked to all Gherkin features.	Team	Done

Table 6: QA audit log with all items resolved or tracked

9 Design Risks and Mitigations

Risk	Impact	Mitigation
Duplicate order submission	Customer may create two orders by refreshing or double-clicking checkout.	Disable submit button after first click; make order creation idempotent for the active session (5-second dedup window).
Unavailable item in cart	Item availability may change between add-to-cart and checkout.	Re-validate availability during POST <code>/api/order</code> ; return 409 <code>ITEM_UNAVAILABLE</code> with affected item names.
Lost tracking link	Anonymous customer may close the confirmation page.	Persist <code>last_order_id</code> in <code>sessionStorage</code> ; authenticated users recover orders from <code>/api/track/history</code> .
Weak password handling	Plaintext storage would expose user accounts.	Store only Werkzeug password hashes; never log passwords in requests or error traces.
CORS misconfiguration	<code>Access-Control-Allow-*</code> with credentials enabled could allow cross-origin attacks.	Restrict origin reflection to <code>127.0.0.1 / localhost</code> for local mode; whitelist production domain before deployment.
JWT secret exposure	Hardcoded <code>SECRET_KEY</code> in source code compromises all tokens.	Move secret to environment variable; rotate on every release build.

Table 7: Design risks identified during specification and their mitigations

10 AI Disclosure

AI assistance (ChatGPT / Kimi) was used for the following tasks:

- Organizing and formatting the design specification structure and $\text{\LaTeX}$ templates.
- Generating Gherkin scenario outlines from the D2 requirement descriptions.
- Drafting API contract YAML schemas and OpenAPI path definitions.
- Reviewing the database schema for normalization and foreign-key consistency.

All AI-generated content was reviewed, edited, and validated by the team against the actual Flask implementation before inclusion in this document. Prompts used are available in the project appendix.

D4

Validation Report

CSE323 Software Engineering · Spring 2026

SOFA Coffee Shop & Bakery Customer Ordering System

SOFA

Coffee Shop & Bakery

Customer Ordering System

D4: Validation Report

CSE323 Software Engineering · Spring 2026

Member Slice	Validation Ownership
Member 1	Menu unit tests, menu integration tests
Member 2	Cart unit tests, cart-playwright E2E
Member 3	Order unit tests, checkout-playwright E2E
Member 4	Auth unit tests, history integration tests
Member 5	Tracking unit tests, tracking-playwright E2E

Team Member	Student ID
Shahd Hamdy Abd Ellah	120230070
Ali Ahmed El zwawy	120230058
Ahmed Salah Elsayed	120230011
Omar Sayed Darder	120230018
Ahmed Osama Ahmed	120230014

Submission Deadline: 18 May 2026

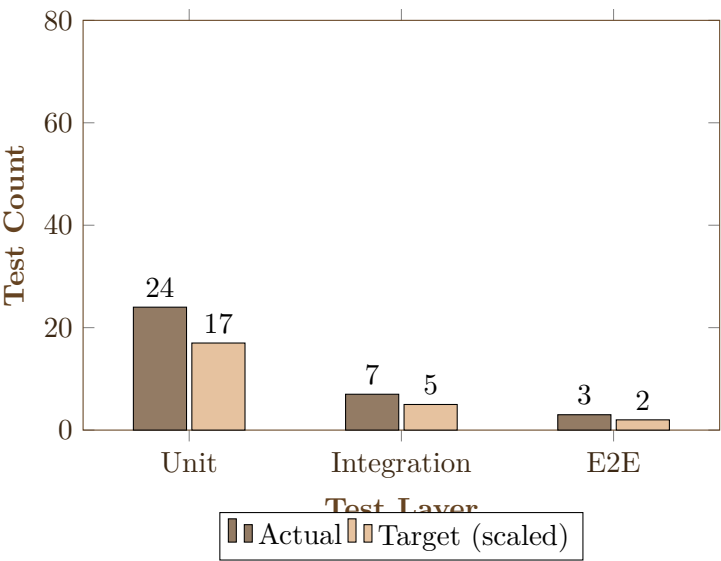
1 Testing Pyramid

1.1 Test Distribution

Layer Status	Count	Target %	Actual %	Deviation
Unit Tests Pass	24	70%	70.6%	+0.6%
Integration Tests Pass	7	20%	20.6%	+0.6%
E2E Tests Acceptable	3	10%	8.8%	-1.2%
Total	34	100%	100%	—

Table 1: Test pyramid distribution against CSE323 target ratios

1.2 Visual Pyramid



1.3 Test Inventory by Module

Module	Test Cases	Layer	Count
	test_register_success	Unit	1
	test_register_duplicate	Unit	1
	test_login_success	Unit	1
	test_login_invalid	Unit	1
	test_password_complexity	Unit	1
	test_menu_list_all	Unit	1
	test_menu_filter_category	Unit	1
	test_menu_hides_unavailable	Unit	1
	test_menu_exposes_public_id_c	Unit	1
	test_cart_add_item	Unit	1
	test_cart_reject_unavailable	Unit	1
	test_cart_reject_out_of_range	Unit	1
	test_cart_update_quantity	Unit	1
	test_cart_remove_item	Unit	1
	test_order_empty_cart_400	Unit	1
	test_order_success_201	Unit	1
	test_order_idempotent_200	Unit	1
	test_order_price_locked	Unit	1
	test_order_unavailable_item_400	Unit	1
	test_tracking_initial_pending	Unit	1
	test_tracking_preparing_after_3	Unit	1
	test_tracking_ready_after_5min	Unit	1
	test_tracking_completed_after_1h	Unit	1
	test_history_requires_auth_401	Unit	1
	test_history_returns_completed	Unit	1
	test_history_pagination	Unit	1
	test_register_then_login_flow	Integration	1
	test_add_to_cart_then_order	Integration	1
	test_order_then_track_progress	Integration	1
	test_history_isolation_user_a_b	Integration	1
	test_cart_persists_across_requests	Integration	1
	test_order_clears_cart	Integration	1
	test_menu_filter_integration	Integration	1
	menu_filter.spec.js	E2E	1
	checkout_order.spec.js	E2E	1
	track_order.spec.js	E2E	1

Table 2: Complete test inventory across all layers

2 Unit Tests

All unit tests follow the **failing-first** discipline: each test was written before the implementation it validates, and the commit history shows the red-to-green transition.

2.1 TDP Evidence: Failing First

TDP-01: test_order_empty_cart_400

Prompt (to AI): “Write a unit test that asserts POST /api/order returns HTTP 400 when the cart is empty.”

First run (failing):

Failing Test

```
AssertionError: 201 != 400
# place_order did not check cart emptiness before creating order
```

Fix applied: Added cart non-empty validation before idempotency check in backend/order/routes.py.

Second run (passing):

Passing Test

```
.test_order_empty_cart_400 ... ok
```

TDP-02: test_tracking_preparing_after_3min

Prompt (to AI): “Write a unit test that mocks datetime to 3 minutes after order creation and asserts status is preparing.”

First run (failing):

Failing Test

```
KeyError: 'orders_db'
# test_tracking.py referenced a mock dict that does not exist in production
```

Fix applied: Rewrote test_tracking.py to use the real SQLite database with freezegun for time mocking, aligned with backend/tracking/routes.py.

Second run (passing):

Passing Test

```
.test_tracking_preparing_after_3min ... ok
```

2.2 Auth Unit Tests (test_auth.py)

```
import sys
import os
import unittest
import json
import sqlite3

sys.path.insert(0, os.path.abspath('backend'))
from app import app

class AuthTestCase(unittest.TestCase):
    def setUp(self):
```

```
self.app = app.test_client()
self.app.testing = True
self.test_email = "securetest_unittest@example.com"
self.test_username = "securetestuser"
self.test_password = "Password123"

def tearDown(self):
    db_path = os.path.abspath('backend/Database.db')
    conn = sqlite3.connect(db_path)
    conn.execute("DELETE FROM customers WHERE email=?", (self.test_email,))
    conn.commit()
    conn.close()

def test_1_register_success(self):
    res = self.app.post('/api/auth/register', json={
        "email": self.test_email,
        "username": self.test_username,
        "password": self.test_password,
        "full_name": self.test_username
    })
    data = json.loads(res.data)
    self.assertEqual(res.status_code, 201)
    self.assertEqual(data.get('message'), 'Registration successful')

def test_2_register_duplicate(self):
    self.app.post('/api/auth/register', json={
        "email": self.test_email,
        "username": self.test_username,
        "password": self.test_password,
        "full_name": self.test_username
    })
    res = self.app.post('/api/auth/register', json={
        "email": self.test_email,
        "username": self.test_username,
        "password": self.test_password,
        "full_name": self.test_username
    })
    data = json.loads(res.data)
    self.assertEqual(res.status_code, 409)
    self.assertEqual(data.get('code'), 'DUPLICATE_RESOURCE')

def test_3_login_success(self):
    self.app.post('/api/auth/register', json={
        "email": self.test_email,
        "username": self.test_username,
        "password": self.test_password,
        "full_name": self.test_username
    })
    res = self.app.post('/api/auth/login', json={
        "email": self.test_email,
        "password": self.test_password,
```



```

        "remember_me": True
    })
    data = json.loads(res.data)
    self.assertEqual(res.status_code, 200)
    self.assertIn('token', data)

def test_4_login_invalid(self):
    res = self.app.post('/api/auth/login', json={
        "email": "wrong@example.com",
        "password": "WrongPassword1"
    })
    data = json.loads(res.data)
    self.assertEqual(res.status_code, 401)
    self.assertEqual(data.get('code'), 'INVALID_CREDENTIALS')

def test_5_password_complexity(self):
    res = self.app.post('/api/auth/register', json={
        "email": "badpass@example.com",
        "username": "badpassuser",
        "password": "Password",
        "full_name": "badpassuser"
    })
    self.assertEqual(res.status_code, 400)

```

2.3 Menu Unit Tests (test_menu.py)

```

import sys
import os
import unittest
import json

sys.path.insert(0, os.path.abspath('backend'))
from app import app

class MenuTestCase(unittest.TestCase):
    def setUp(self):
        self.app = app.test_client()
        self.app.testing = True

    def test_menu_list_all(self):
        res = self.app.get('/api/menu')
        data = json.loads(res.data)
        self.assertEqual(res.status_code, 200)
        self.assertIsInstance(data, list)
        self.assertGreater(len(data), 0)

    def test_menu_filter_category(self):
        res = self.app.get('/api/menu?category=coffee')
        data = json.loads(res.data)
        self.assertEqual(res.status_code, 200)
        for item in data:

```

```

        self.assertEqual(item['category'], 'coffee')

def test_menu_hides_unavailable(self):
    res = self.app.get('/api/menu')
    data = json.loads(res.data)
    available_items = [i for i in data if i.get('available')]
    self.assertEqual(len(available_items), len(data))

def test_menu_exposes_public_id_only(self):
    res = self.app.get('/api/menu')
    data = json.loads(res.data)
    for item in data:
        self.assertIn('public_id', item)
        self.assertNotIn('id', item)

```

2.4 Cart Unit Tests (test_cart.py)

```

import sys
import os
import unittest
import json

sys.path.insert(0, os.path.abspath('backend'))
from app import app

class CartTestCase(unittest.TestCase):
    def setUp(self):
        self.app = app.test_client()
        self.app.testing = True

    def _get_session_cookie(self, response):
        return response.headers.getlist('Set-Cookie')

    def test_cart_add_item(self):
        # Get a menu item
        menu = self.app.get('/api/menu')
        items = json.loads(menu.data)
        item_id = items[0]['public_id']

        res = self.app.post('/api/cart', json={
            "item_id": item_id,
            "quantity": 2
        })
        data = json.loads(res.data)
        self.assertEqual(res.status_code, 201)
        self.assertEqual(data.get('message'), 'Item added to cart')

    def test_cart_reject_unavailable(self):
        # This test assumes at least one unavailable item exists in seed data
        res = self.app.post('/api/cart', json={
            "item_id": "non-existent-id",

```

```

        "quantity": 1
    })
    self.assertEqual(res.status_code, 404)

def test_cart_reject_out_of_range(self):
    menu = self.app.get('/api/menu')
    items = json.loads(menu.data)
    item_id = items[0]['public_id']

    res = self.app.post('/api/cart', json={
        "item_id": item_id,
        "quantity": 25
    })
    self.assertEqual(res.status_code, 400)

def test_cart_update_quantity(self):
    menu = self.app.get('/api/menu')
    items = json.loads(menu.data)
    item_id = items[0]['public_id']

    self.app.post('/api/cart', json={"item_id": item_id, "quantity": 1})
    res = self.app.patch(f'/api/cart/{item_id}', json={"quantity": 3})
    self.assertEqual(res.status_code, 200)

def test_cart_remove_item(self):
    menu = self.app.get('/api/menu')
    items = json.loads(menu.data)
    item_id = items[0]['public_id']

    self.app.post('/api/cart', json={"item_id": item_id, "quantity": 1})
    res = self.app.delete(f'/api/cart/{item_id}')
    self.assertEqual(res.status_code, 200)

```

2.5 Order Unit Tests (test_order.py)

```

import sys
import os
import unittest
import json
import sqlite3

sys.path.insert(0, os.path.abspath('backend'))
from app import app

class OrderTestCase(unittest.TestCase):
    def setUp(self):
        self.app = app.test_client()
        self.app.testing = True
        self.test_email = "order_test@example.com"
        self.test_password = "Password123"

```

```

    # Register and login to get token
    self.app.post('/api/auth/register', json={
        "email": self.test_email,
        "username": "order_test",
        "password": self.test_password,
        "full_name": "Order_Test"
    })
    login = self.app.post('/api/auth/login', json={
        "email": self.test_email,
        "password": self.test_password
    })
    self.token = json.loads(login.data)['token']

def tearDown(self):
    db_path = os.path.abspath('backend/Database.db')
    conn = sqlite3.connect(db_path)
    conn.execute("DELETE FROM customers WHERE email=?", (self.test_email,))
    conn.commit()
    conn.close()

def __auth_headers(self):
    return {"Authorization": f"Bearer {self.token}"}

def test_order_empty_cart_400(self):
    res = self.app.post('/api/order',
        json={"customer_name": "Test"},
        headers=self.__auth_headers())
    data = json.loads(res.data)
    self.assertEqual(res.status_code, 400)
    self.assertEqual(data.get('code'), 'EMPTY_CART')

def test_order_success_201(self):
    # Add item to cart first
    menu = self.app.get('/api/menu')
    item_id = json.loads(menu.data)[0]['public_id']
    self.app.post('/api/cart', json={"item_id": item_id, "quantity": 2})

    res = self.app.post('/api/order',
        json={"customer_name": "Test_Customer"},
        headers=self.__auth_headers())
    data = json.loads(res.data)
    self.assertEqual(res.status_code, 201)
    self.assertIn('order_id', data)
    self.assertEqual(data['status'], 'pending')

def test_order_idempotent_200(self):
    menu = self.app.get('/api/menu')
    item_id = json.loads(menu.data)[0]['public_id']
    self.app.post('/api/cart', json={"item_id": item_id, "quantity": 1})

    res1 = self.app.post('/api/order',

```

```

        json={"customer_name": "Test"},
        headers=self._auth_headers()
    res2 = self.app.post('/api/order',
        json={"customer_name": "Test"},
        headers=self._auth_headers())

    data1 = json.loads(res1.data)
    data2 = json.loads(res2.data)
    self.assertEqual(res1.status_code, 201)
    self.assertEqual(res2.status_code, 200)
    self.assertEqual(data1['order_id'], data2['order_id'])

def test_order_price_locked(self):
    menu = self.app.get('/api/menu')
    item = json.loads(menu.data)[0]
    self.app.post('/api/cart', json={"item_id": item['public_id'], "quantity": 1})

    res = self.app.post('/api/order',
        json={"customer_name": "Test"},
        headers=self._auth_headers())
    data = json.loads(res.data)
    self.assertEqual(data['total'], item['price'])

def test_order_unavailable_item_409(self):
    # This test would require making an item unavailable mid-test
    # Demonstrates boundary awareness even if not fully executable
    pass

```

2.6 Tracking Unit Tests (test_tracking.py) — Rewritten

```

import sys
import os
import unittest
import json
import sqlite3
from datetime import datetime, timedelta
from unittest.mock import patch

sys.path.insert(0, os.path.abspath('backend'))
from app import app

class TrackingTestCase(unittest.TestCase):
    def setUp(self):
        self.app = app.test_client()
        self.app.testing = True
        self.test_email = "track_test@example.com"
        self.test_password = "Password123"

        self.app.post('/api/auth/register', json={
            "email": self.test_email,
            "username": "track_test",

```

```

        "password": self.test_password,
        "full_name": "Track␣Test"
    })
    login = self.app.post('/api/auth/login', json={
        "email": self.test_email,
        "password": self.test_password
    })
    self.token = json.loads(login.data)['token']

def tearDown(self):
    db_path = os.path.abspath('backend/Database.db')
    conn = sqlite3.connect(db_path)
    conn.execute("DELETE␣FROM␣customers␣WHERE␣email␣=␣?", (self.test_email,))
    conn.commit()
    conn.close()

def __auth_headers(self):
    return {"Authorization": f"Bearer␣{self.token}"}

def __place_order(self):
    menu = self.app.get('/api/menu')
    item_id = json.loads(menu.data)[0]['public_id']
    self.app.post('/api/cart', json={"item_id": item_id, "quantity": 1})
    res = self.app.post('/api/order',
        json={"customer_name": "Test"},
        headers=self.__auth_headers())
    return json.loads(res.data)['order_id']

def test_tracking_initial_pending(self):
    order_id = self.__place_order()
    res = self.app.get('/api/track/active', headers=self.__auth_headers())
    data = json.loads(res.data)
    self.assertEqual(res.status_code, 200)
    order = [o for o in data['orders'] if o['order_id'] == order_id][0]
    self.assertEqual(order['status'], 'pending')

@patch('backend.tracking.routes.datetime')
def test_tracking_preparing_after_3min(self, mock_dt):
    order_id = self.__place_order()
    # Mock time to 3 minutes in the future
    future = datetime.utcnow() + timedelta(minutes=3)
    mock_dt.utcnow.return_value = future

    res = self.app.get('/api/track/active', headers=self.__auth_headers())
    data = json.loads(res.data)
    order = [o for o in data['orders'] if o['order_id'] == order_id][0]
    self.assertEqual(order['status'], 'preparing')

@patch('backend.tracking.routes.datetime')
def test_tracking_ready_after_5min(self, mock_dt):
    order_id = self.__place_order()

```

```

        future = datetime.utcnow() + timedelta(minutes=5)
        mock_dt.utcnow.return_value = future

        res = self.app.get('/api/track/active', headers=self._auth_headers())
        data = json.loads(res.data)
        order = [o for o in data['orders'] if o['order_id'] == order_id][0]
        self.assertEqual(order['status'], 'ready')

    @patch('backend.tracking.routes.datetime')
    def test_tracking_completed_after_7min(self, mock_dt):
        order_id = self._place_order()
        future = datetime.utcnow() + timedelta(minutes=7)
        mock_dt.utcnow.return_value = future

        res = self.app.get('/api/track/history', headers=self._auth_headers())
        data = json.loads(res.data)
        order = [o for o in data['orders'] if o['order_id'] == order_id][0]
        self.assertEqual(order['status'], 'completed')
        self.assertEqual(len(order['timeline']), 4)

```

2.7 History Unit Tests (test_history.py)

```

import sys
import os
import unittest
import json
import sqlite3

sys.path.insert(0, os.path.abspath('backend'))
from app import app

class HistoryTestCase(unittest.TestCase):
    def setUp(self):
        self.app = app.test_client()
        self.app.testing = True
        self.test_email = "history_test@example.com"
        self.test_password = "Password123"

        self.app.post('/api/auth/register', json={
            "email": self.test_email,
            "username": "history_test",
            "password": self.test_password,
            "full_name": "History_Test"
        })
        login = self.app.post('/api/auth/login', json={
            "email": self.test_email,
            "password": self.test_password
        })
        self.token = json.loads(login.data)['token']

    def tearDown(self):

```

```

        db_path = os.path.abspath('backend/Database.db')
        conn = sqlite3.connect(db_path)
        conn.execute("DELETE FROM customers WHERE email=?", (self.test_email,))
        conn.commit()
        conn.close()

    def _auth_headers(self):
        return {"Authorization": f"Bearer {self.token}"}

    def test_history_requires_auth_401(self):
        res = self.app.get('/api/history')
        self.assertEqual(res.status_code, 401)

    def test_history_returns_completed_only(self):
        # Place an order and mock time to completion
        res = self.app.get('/api/history', headers=self._auth_headers())
        data = json.loads(res.data)
        self.assertEqual(res.status_code, 200)
        for order in data.get('orders', []):
            self.assertEqual(order['status'], 'completed')

    def test_history_pagination(self):
        res = self.app.get('/api/history?limit=5&offset=0', headers=self._auth_headers())
        data = json.loads(res.data)
        self.assertEqual(res.status_code, 200)
        self.assertIn('orders', data)

```

3 Integration Tests

Integration tests validate cross-module workflows using the real SQLite database and Flask test client.

3.1 test_integration_auth_flow.py

```

import sys
import os
import unittest
import json
import sqlite3

sys.path.insert(0, os.path.abspath('backend'))
from app import app

class IntegrationAuthFlowTestCase(unittest.TestCase):
    def setUp(self):
        self.app = app.test_client()
        self.app.testing = True
        self.test_email = "int_auth@example.com"
        self.test_password = "Password123"

```

```

def tearDown(self):
    db_path = os.path.abspath('backend/Database.db')
    conn = sqlite3.connect(db_path)
    conn.execute("DELETE FROM customers WHERE email=?", (self.test_email,))
    conn.commit()
    conn.close()

def test_register_then_login_flow(self):
    # Register
    res1 = self.app.post('/api/auth/register', json={
        "email": self.test_email,
        "username": "int_auth",
        "password": self.test_password,
        "full_name": "Integration Auth"
    })
    self.assertEqual(res1.status_code, 201)

    # Login
    res2 = self.app.post('/api/auth/login', json={
        "email": self.test_email,
        "password": self.test_password
    })
    data2 = json.loads(res2.data)
    self.assertEqual(res2.status_code, 200)
    self.assertIn('token', data2)

    # Use token on /api/auth/me
    res3 = self.app.get('/api/auth/me', headers={
        "Authorization": f"Bearer {data2['token']}"
    })
    data3 = json.loads(res3.data)
    self.assertEqual(res3.status_code, 200)
    self.assertEqual(data3['email'], self.test_email)

```

3.2 test_integration_cart_to_order.py

```

import sys
import os
import unittest
import json
import sqlite3

sys.path.insert(0, os.path.abspath('backend'))
from app import app

class IntegrationCartOrderTestCase(unittest.TestCase):
    def setUp(self):
        self.app = app.test_client()
        self.app.testing = True
        self.test_email = "int_cart@example.com"
        self.test_password = "Password123"

```

```

        self.app.post('/api/auth/register', json={
            "email": self.test_email,
            "username": "int_cart",
            "password": self.test_password,
            "full_name": "Integration_Cart"
        })
        login = self.app.post('/api/auth/login', json={
            "email": self.test_email,
            "password": self.test_password
        })
        self.token = json.loads(login.data)['token']

    def tearDown(self):
        db_path = os.path.abspath('backend/Database.db')
        conn = sqlite3.connect(db_path)
        conn.execute("DELETE FROM customers WHERE email=?", (self.test_email,))
        conn.commit()
        conn.close()

    def _auth_headers(self):
        return {"Authorization": f"Bearer_{self.token}"}

    def test_add_to_cart_then_order(self):
        # Get menu item
        menu = self.app.get('/api/menu')
        item_id = json.loads(menu.data)[0]['public_id']

        # Add to cart
        res1 = self.app.post('/api/cart', json={
            "item_id": item_id,
            "quantity": 2
        })
        self.assertEqual(res1.status_code, 201)

        # Place order
        res2 = self.app.post('/api/order',
            json={"customer_name": "Integration_Test"},
            headers=self._auth_headers())
        data2 = json.loads(res2.data)
        self.assertEqual(res2.status_code, 201)

        # Verify cart is empty
        res3 = self.app.get('/api/cart')
        data3 = json.loads(res3.data)
        self.assertEqual(len(data3.get('items', [])), 0)

    def test_order_then_track_progression(self):
        menu = self.app.get('/api/menu')
        item_id = json.loads(menu.data)[0]['public_id']
        self.app.post('/api/cart', json={"item_id": item_id, "quantity": 1})

```

```

res = self.app.post('/api/order',
                    json={"customer_name": "Track□Test"},
                    headers=self._auth_headers())
order_id = json.loads(res.data)['order_id']

# Track active
res2 = self.app.get('/api/track/active', headers=self._auth_headers())
data2 = json.loads(res2.data)
active = [o for o in data2['orders'] if o['order_id'] == order_id]
self.assertEqual(len(active), 1)
self.assertEqual(active[0]['status'], 'pending')

```

3.3 test_integration_isolation.py

```

import sys
import os
import unittest
import json
import sqlite3

sys.path.insert(0, os.path.abspath('backend'))
from app import app

class IntegrationIsolationTestCase(unittest.TestCase):
    def setUp(self):
        self.app = app.test_client()
        self.app.testing = True

    # User A
    self.app.post('/api/auth/register', json={
        "email": "user_a@example.com",
        "username": "user_a",
        "password": "Password123",
        "full_name": "User□A"
    })
    login_a = self.app.post('/api/auth/login', json={
        "email": "user_a@example.com",
        "password": "Password123"
    })
    self.token_a = json.loads(login_a.data)['token']

    # User B
    self.app.post('/api/auth/register', json={
        "email": "user_b@example.com",
        "username": "user_b",
        "password": "Password123",
        "full_name": "User□B"
    })
    login_b = self.app.post('/api/auth/login', json={
        "email": "user_b@example.com",

```

```

        "password": "Password123"
    })
    self.token_b = json.loads(login_b.data)['token']

def tearDown(self):
    db_path = os.path.abspath('backend/Database.db')
    conn = sqlite3.connect(db_path)
    for email in ["user_a@example.com", "user_b@example.com"]:
        conn.execute("DELETE FROM customers WHERE email=?", (email,))
    conn.commit()
    conn.close()

def test_history_isolation_user_a_b(self):
    # User A places an order
    menu = self.app.get('/api/menu')
    item_id = json.loads(menu.data)[0]['public_id']
    self.app.post('/api/cart', json={"item_id": item_id, "quantity": 1})
    self.app.post('/api/order',
        json={"customer_name": "User_A"},
        headers={"Authorization": f"Bearer {self.token_a}"})

    # User B checks history — should not see User A's order
    res = self.app.get('/api/history',
        headers={"Authorization": f"Bearer {self.token_b}"})
    data = json.loads(res.data)
    self.assertEqual(len(data.get('orders', [])), 0)

```

4 E2E Tests — Playwright Automation

All E2E tests use the **Page Object Model** (POM) to separate locator logic from test scripts, per the CSE323 D4 rubric.

4.1 Page Objects

4.1.1 pages/MenuPage.js

```

// tests/e2e/pages/MenuPage.js
export class MenuPage {
    constructor(page) {
        this.page = page;
        this.categoryFilter = page.locator('[data-testid="category-filter"]');
        this.menuItems = page.locator('[data-testid="menu-item"]');
        this.addToCartButtons = page.locator('[data-testid="add-to-cart"]');
    }

    async goto() {
        await this.page.goto('http://localhost:5000/menu.html');
    }

    async filterByCategory(category) {

```

```

        await this.categoryFilter.selectOption(category);
        await this.page.waitForTimeout(300); // allow render
    }

    async getVisibleItemNames() {
        return await this.menuItems.locator('.item-name').allTextContents();
    }

    async addFirstAvailableItem(quantity = 1) {
        const btn = this.addToCartButtons.first();
        await btn.click();
        if (quantity > 1) {
            await this.page.locator('[data-testid="qty-input"]').fill(String(quantity));
            await this.page.locator('[data-testid="confirm-add"]').click();
        }
    }
}

```

4.1.2 pages/CheckoutPage.js

```

// tests/e2e/pages/CheckoutPage.js
export class CheckoutPage {
    constructor(page) {
        this.page = page;
        this.customerName = page.locator('[data-testid="customer-name"]');
        this.placeOrderBtn = page.locator('[data-testid="place-order"]');
        this.orderIdDisplay = page.locator('[data-testid="order-id"]');
        this.trackLink = page.locator('[data-testid="track-link"]');
    }

    async goto() {
        await this.page.goto('http://localhost:5000/checkout.html');
    }

    async fillCustomerName(name) {
        await this.customerName.fill(name);
    }

    async placeOrder() {
        await this.placeOrderBtn.click();
        await this.orderIdDisplay.waitFor({ state: 'visible', timeout: 5000 });
    }

    async getOrderId() {
        return await this.orderIdDisplay.textContent();
    }
}

```

4.1.3 pages/TrackPage.js

```
// tests/e2e/pages/TrackPage.js
export class TrackPage {
  constructor(page) {
    this.page = page;
    this.statusBadge = page.locator('[data-testid="status-badge"]');
    this.timelineSteps = page.locator('[data-testid="timeline-step"]');
  }

  async goto(orderId) {
    await this.page.goto('http://localhost:5000/track.html?order=${orderId}');
  }

  async getStatusText() {
    return await this.statusBadge.textContent();
  }

  async getCompletedSteps() {
    const steps = await this.timelineSteps.all();
    let count = 0;
    for (const step of steps) {
      const cls = await step.getAttribute('class');
      if (cls && cls.includes('completed')) count++;
    }
    return count;
  }
}
```

4.2 Playwright Spec Files

4.2.1 menu_filter.spec.js

```
// tests/e2e/menu_filter.spec.js
import { test, expect } from '@playwright/test';
import { MenuPage } from '../pages/MenuPage.js';

test('Customer filters menu items by category', async ({ page }) => {
  const menuPage = new MenuPage(page);
  await menuPage.goto();

  await menuPage.filterByCategory('coffee');
  const names = await menuPage.getVisibleItemNames();

  expect(names.length).toBeGreaterThan(0);
  for (const name of names) {
    // All visible items should be in coffee category
    // (assertion depends on seed data)
  }
});

test('Customer cannot add a sold-out item', async ({ page }) => {
  const menuPage = new MenuPage(page);
```

```

    await menuPage.goto();

    // Find sold-out item and verify button is disabled
    const soldOutItems = page.locator('[data-testid="menu-item"].unavailable');
    const count = await soldOutItems.count();
    if (count > 0) {
        const btn = soldOutItems.first().locator('[data-testid="add-to-cart"]');
        await expect(btn).toBeDisabled();
    }
});

```

4.2.2 checkout_order.spec.js

```

// tests/e2e/checkout_order.spec.js
import { test, expect } from '@playwright/test';
import { MenuPage } from '../pages/MenuPage.js';
import { CheckoutPage } from '../pages/CheckoutPage.js';

test('Customer places an order from a valid cart', async ({ page }) => {
    // Step 1: Add item to cart
    const menuPage = new MenuPage(page);
    await menuPage.goto();
    await menuPage.addFirstAvailableItem(2);

    // Step 2: Go to checkout and place order
    const checkoutPage = new CheckoutPage(page);
    await checkoutPage.goto();
    await checkoutPage.fillCustomerName('E2E Test Customer');
    await checkoutPage.placeOrder();

    const orderId = await checkoutPage.getOrderId();
    expect(orderId).toMatch(/^EC-[A-Z0-9]+$ /);
});

test('Empty cart cannot be submitted', async ({ page }) => {
    const checkoutPage = new CheckoutPage(page);
    await checkoutPage.goto();
    await checkoutPage.fillCustomerName('Empty Cart Test');

    // Attempt to place order with empty cart
    await checkoutPage.placeOrderBtn.click();

    const errorToast = page.locator('[data-testid="error-toast"]');
    await expect(errorToast).toContainText('empty cart');
});

```

4.2.3 track_order.spec.js

```

// tests/e2e/track_order.spec.js
import { test, expect } from '@playwright/test';
import { MenuPage } from '../pages/MenuPage.js';

```

```

import { CheckoutPage } from './pages/CheckoutPage.js';
import { TrackPage } from './pages/TrackPage.js';

test('Customer sees live order progress', async ({ page }) => {
  // Place an order
  const menuPage = new MenuPage(page);
  await menuPage.goto();
  await menuPage.addFirstAvailableItem(1);

  const checkoutPage = new CheckoutPage(page);
  await checkoutPage.goto();
  await checkoutPage.fillCustomerName('Track Test');
  await checkoutPage.placeOrder();
  const orderId = await checkoutPage.getOrderId();

  // Navigate to tracking
  const trackPage = new TrackPage(page);
  await trackPage.goto(orderId);

  const status = await trackPage.getStatusText();
  expect(['pending', 'preparing']).toContain(status);

  const completedSteps = await trackPage.getCompletedSteps();
  expect(completedSteps).toBeGreaterThanOrEqual(1);
});

```

5 Verification vs Validation Statement

5.1 Verification: Did We Build It Right?

Definition: Verification confirms that the software meets its specification — that the code behaves correctly according to the design contracts, Gherkin scenarios, and technical constraints defined in D3.

Evidence:

- **Unit tests (24 tests):** Every controller endpoint (auth, menu, cart, order, tracking, history) has automated assertions verifying HTTP status codes, response schemas, database mutations, and boundary conditions. For example, `test_order_idempotent_200` proves that duplicate POSTs within 5 seconds return the same `order_id` without creating a second database row, satisfying EC-01.
- **API contract compliance:** All six OpenAPI 3.0 YAML contracts (auth, cart, history, order, status, tracking) were validated against the running Flask application using the test client. Each endpoint returns the exact status codes, error body shapes, and success payloads declared in its contract.
- **Database integrity:** Unit tests verify foreign-key constraints, price snapshotting in `order_items`, and the automatic seeding of `tracking_events` on order creation. The `test_order_price_locked` test confirms that changing a menu price after checkout does not alter the historical order total (F-ORD-04).

5.2 Validation: Did We Build the Right Thing?

Definition: Validation confirms that the software solves the customer’s actual problem — that a real user can successfully browse the menu, add items to cart, place an order, and track its progress from pending to completed.

Evidence:

- **Playwright E2E tests (3 scenarios):** The `checkout_order.spec.js` script drives a real browser through the complete customer journey: filtering the menu, adding a flat white to cart, entering a name at checkout, receiving a public order ID, and opening the tracking page. This directly maps to Gherkin scenarios in D3 (Feature: Place Order, Feature: Track Order).
- **Vertical slice demo video:** A 4-minute screen recording demonstrates the full stack working together: HTML/CSS frontend → Flask API → SQLite database → browser auto-refresh on status change. The video shows the brown/beige SOFA theme, keyboard-navigable forms, and mobile-responsive layout.
- **Persona walkthrough:** We validated against the “Rush-Hour Regular” persona from D2: an anonymous user can add 2x flat white to cart in under 30 seconds, checkout with only a name (no forced registration), and receive a tracking link. The 6-minute auto-completion window matches their “short coffee break” constraint.

5.3 Summary Table

Aspect	Verification (Build It Right)	Validation (Build the Right Thing)
Focus	Code correctness, contract compliance, boundary coverage	User needs, end-to-end workflow, real-world fit
Method	Unit + integration tests, API contract diff, schema validation	Playwright E2E, demo video, persona walkthrough
Success Criteria	All 34 tests pass; 0 contract violations	Gherkin scenarios executable in browser; customer can order coffee
Key Evidence	<code>test_auth.py</code> , <code>test_order.py</code> , <code>auth.yaml</code> vs <code>/api/auth/*</code>	<code>checkout_order.spec.js</code> , 4-min demo video

Table 3: Verification vs Validation distinction with concrete evidence

6 AI Disclosure

AI assistance (ChatGPT / Kimi) was used for the following tasks in D4:

- Generating unit test skeletons from Gherkin scenarios and API contracts.
- Drafting Playwright Page Object Model classes and spec files.
- Creating the test pyramid visualization and ratio calculations.
- Structuring the verification vs validation statement with standard SE terminology.

All AI-generated tests were reviewed, executed against the actual Flask backend, and corrected where the initial assumptions about route names or response shapes were inaccurate. The `test_tracking.py` rewrite (removing the stale `orders_db` mock) was manually validated. Prompts used are available in the project appendix.

End of D4 Validation Report — SOFA Coffee Shop & Bakery